Monte Carlo Tree Search for 3D Connect Four: Tackling Combinatorial Explosion with Intelligent Sampling

Caden Calderon

Dec, 2025

Abstract

This report presents the design and implementation of an AI opponent for Axial, a three-dimensional variant of Connect Four played on a $6 \times 6 \times 7$ grid. The game features a gesture-based interface where players interact using hand tracking through a standard webcam, with the AI running as a separate Python backend communicating via TCP sockets. Traditional game-solving techniques such as Minimax with alpha-beta pruning fail at this scale due to the combinatorial explosion of approximately 10^{120} possible board states. To address this challenge, we implement Monte Carlo Tree Search (MCTS) enhanced with Upper Confidence Bounds for Trees (UCT), Rapid Action Value Estimation (RAVE), and a tactical threat detection layer. The system is further optimized using Numba JIT compilation, achieving approximately 619 simulations per second. Experimental evaluation demonstrates a 100% win rate against both random and greedy baseline opponents, validating the effectiveness of our approach for complex 3D game environments.

1 Introduction

Connect Four is a classic two-player strategy game where players alternate dropping colored pieces into a vertical grid, with pieces falling to the lowest available position due to gravity. The objective is to connect four pieces in a row. While standard Connect Four on a 7×6 board has been completely solved [3], extending the game into three dimensions creates a vastly more complex challenge.

This report presents Axial, a 3D Connect Four game with gesture-based hand tracking controls, and focuses on the AI system that powers its computer opponent. The following sections first review related work in game-playing AI, then describe the game itself, explain the technical challenges that motivated our AI approach, and finally detail the Monte Carlo Tree Search implementation that addresses those challenges.

2 Related Work

2.1 Monte Carlo Tree Search

MCTS emerged as a breakthrough for game-playing AI when it achieved strong amateur-level play in Go without domain-specific knowledge [5]. The algorithm builds an asymmetric game tree by repeatedly sampling random playouts and backpropagating results.

2.2 Upper Confidence Bounds for Trees (UCT)

Kocsis and Szepesvári [2] introduced UCT, applying the Upper Confidence Bound bandit algorithm to tree search. UCT provides theoretical convergence guarantees and balances exploitation of known good moves with exploration of less-visited branches.

2.3 Rapid Action Value Estimation (RAVE)

Gelly and Silver [4] proposed RAVE to address the cold-start problem in MCTS. RAVE shares action value estimates across tree positions, assuming a move's quality is somewhat context-independent. This allows new nodes to benefit from experience gained elsewhere.

2.4 AlphaZero

Silver et al. [1] demonstrated that MCTS combined with deep neural networks could achieve superhuman performance in Chess, Shogi, and Go through self-play. While our implementation does not use neural networks, we draw inspiration from AlphaZero's demonstration that MCTS scales to complex games.

2.5 Connect Four Solutions

Tromp's Fhourstones benchmark [3] provides optimal solutions for standard Connect Four using bitboard techniques. We adapted their 1D array flattening approach for our 3D board representation, enabling efficient memory access patterns compatible with Numba JIT compilation.

3 The Game

3.1 Overview

Axial reimagines Connect Four as a spatial challenge in three dimensions. Two players take turns dropping colored pieces into a transparent volumetric grid, where gravity pulls each piece down to the lowest available position in its column. The first player to align four pieces in a row wins.

3.2 The 3D Board

The game board is a transparent rectangular volume divided into cells:

- **7 columns** along the X-axis (left to right)
- **6 rows** along the Z-axis (front to back, representing depth)
- **6 layers** along the Y-axis (bottom to top, where pieces stack)

This creates $7 \times 6 \times 6 = 252$ individual cells. Each column functions like a vertical tube: when a piece is dropped into column (x, z), it falls to the lowest unoccupied Y position.

3.3 Winning Conditions

A player wins by placing four pieces in a consecutive line. In 3D space, winning lines can form along 13 different directions:

Table 1: Winning Line Directions

Category	Directions
Axial (3)	Along X, Along Y, Along Z
Planar Diagonals (6)	XY, XZ, YZ planes (both orientations each)
Space Diagonals (4)	Through 3D corners (all XYZ combinations)

Compare this to standard 2D Connect Four, which has only 4 directions (horizontal, vertical, and two diagonals). The expansion to 13 directions is what makes 3D Connect Four strategically rich but computationally challenging.

3.4 How to Play

Rather than clicking with a mouse, players interact with Axial using their hands in front of a webcam:

1. **Grab:** Move your hand near a piece until it glows, then pinch (touch thumb to index finger) to pick it up
2. **Position:** While pinching, move your hand to position the piece over your desired column
3. **Drop:** Release the pinch to drop the piece; it falls due to gravity
4. **Rotate:** Use both hands to rotate the board and view it from different angles

Visual feedback guides players: glowing objects indicate grab range, green/red tints show valid/invalid drop locations, and a ghost preview shows where pieces will land.

4 System Architecture

4.1 Two-System Design

Axial is built as two separate programs that communicate over the network:

```
+----------------------------------------+
|           UNITY (C#) - Frontend        |
|  - Renders the 3D game board and pieces |
|  - Processes webcam for hand tracking   |
|  - Handles all player interaction       |
|  - Displays visual feedback             |
+-----------------+----------------------+
                  | TCP/JSON messages
                  | (localhost:5555)
+-----------------+----------------------+
|         PYTHON - AI Backend             |
|  - Runs MCTS algorithm                  |
|  - Manages authoritative game state     |
|  - Validates moves                      |
|  - Determines game outcomes             |
+----------------------------------------+
```

This separation exists because the AI requires heavy computation (thousands of game simulations per move). Running the AI in a separate Python process prevents the game from freezing while it "thinks." Unity maintains smooth 60 FPS rendering, while Python crunches numbers in parallel.

4.2 Communication Protocol

The two systems exchange JSON messages over TCP sockets:

- **Player move:** Unity sends the column where the player dropped their piece
- **AI response:** Python calculates the best move and sends it back
- **Game state:** Status updates include winner detection and draw conditions

4.3 Hand Tracking

The Unity frontend uses Google's MediaPipe to detect hands in the webcam feed. MediaPipe provides 21 landmark points per hand, which are converted into 3D positions for game interaction.

The main technical challenge was inferring depth (how far the hand is from the camera) using only a 2D webcam image. The solution uses apparent hand size as a depth proxy: when a hand is close to the camera it appears large, and when far away it appears small. Multiple smoothing layers eliminate jitter to ensure stable, responsive interaction. Full details of the hand tracking implementation are covered in a companion Game Report.

4.4 Game Flow

A complete turn follows this path:

```
Player drops piece
        |
        v
[GridManager] -- Validates column, animates piece fall
        |
        v
[GameController] -- Sends move to Python via TCP
        |
        v
   [PYTHON AI] -- Runs MCTS, returns best move
        |
        v
[GameController] -- Receives AI response
        |
        v
[GridManager] -- Places AI's piece, checks for win
        |
        v
Player's next turn (or game end)
```

The Python backend maintains the authoritative game state. Even though Unity tracks the board for rendering, Python validates all moves and determines outcomes.

5 The AI Challenge

With the game and system architecture established, we now turn to the core technical challenge: building an AI that can play competitively in this 3D environment.

5.1 Combinatorial Explosion

Standard 2D Connect Four has 42 cells and approximately 4.5×10^{12} possible game states. Axial's 252 cells expand this to approximately:

$$3^{252} \approx 10^{120} \tag{1}$$

To put this in perspective, 10^{120} exceeds estimates for the number of atoms in the observable universe ($\approx 10^{80}$). This combinatorial explosion makes exhaustive search completely infeasible.

5.2 Why Traditional Approaches Fail

The Minimax algorithm with alpha-beta pruning has time complexity $O(b^d)$, where b is the branching factor and d is the search depth. For Axial:

- **Branching factor:** Up to 42 possible moves per turn
- **Required depth:** 20+ plies for strategic play
- **Computation:** $42^{20} \approx 10^{32}$ node evaluations

Even with aggressive pruning, optimized 2D Connect Four solvers struggle beyond depth 8-10. The 3D variant requires a fundamentally different approach.

5.3 Our Solution: Monte Carlo Tree Search

We employ Monte Carlo Tree Search (MCTS), an algorithm that samples the game tree intelligently rather than exhaustively. MCTS has demonstrated success in games with large state spaces, most notably in AlphaGo and AlphaZero [1].

MCTS does not need to evaluate every possible game state. Instead, it:

1. Runs thousands of simulated games (rollouts) from the current position
2. Tracks which moves lead to wins most often
3. Balances exploring new moves versus exploiting known good ones
4. Returns the most-visited move as its choice

Our implementation enhances basic MCTS with:

- UCT (Upper Confidence Bounds for Trees) for balanced exploration/exploitation
- RAVE (Rapid Action Value Estimation) to accelerate learning
- Tactical threat detection to catch critical positions
- Numba JIT compilation for real-time performance

6 Methodology

6.1 Board Representation

The 3D game board is stored as a one-dimensional NumPy array of unsigned 8-bit integers, where each cell contains 0 (empty), 1 (Player 1), or 2 (Player 2). The flat indexing scheme converts 3D coordinates (h, r, c) to a linear index:

$$\text{idx} = h + r \cdot D + c \cdot D \cdot R \tag{2}$$

where $D = 6$ (height), $R = 6$ (rows), and $C = 7$ (columns). This ensures cells within a column are contiguous in memory, optimizing gravity-based piece placement.

6.2 Win Detection

Winning conditions require four consecutive pieces along any of 13 directional vectors:

Table 2: Win Check Direction Vectors

Category	**Direction Vectors** (dh, dr, dc)
Axial (3)	$(1,0,0)$, $(0,1,0)$, $(0,0,1)$
Planar Diagonal (6)	$(1,1,0)$, $(1,-1,0)$, $(0,1,1)$, $(0,1,-1)$, $(1,0,1)$, $(1,0,-1)$
Space Diagonal (4)	$(1,1,1)$, $(1,1,-1)$, $(1,-1,1)$, $(1,-1,-1)$

The algorithm iterates through all board positions and checks for four consecutive pieces in each applicable direction, with bounds checking to ensure we only examine directions where four pieces could fit.

6.3 Monte Carlo Tree Search

MCTS builds a search tree incrementally through repeated simulations. Each simulation consists of four phases: *select* a path through known territory, *expand* by adding one new node, *simulate* a random game to completion, and *backpropagate* the result to update statistics. Over thousands of iterations, frequently-winning moves accumulate more visits, and the most-visited move from the root becomes the AI's choice.

Algorithm 1 MCTS Search Loop

```
function SEARCH(root, simulations)
    for i = 1 to simulations do
        node ← SELECT(root)
        child ← EXPAND(node)
        result ← SIMULATE(child)
        BACKPROPAGATE(child, result)
    end for
    return child of root with highest visit count
end function
```

6.3.1 Selection Phase

Selection traverses from root to leaf, at each node choosing which child to visit next. This choice faces a fundamental tradeoff: should we revisit moves that have performed well (exploitation), or try less-explored moves that might be even better (exploration)? Focusing too much on known-good moves might miss superior alternatives; exploring too broadly wastes simulations on clearly bad moves.

The UCT formula balances this tradeoff mathematically:

$$\text{UCT}(n) = \underbrace{\frac{w_n}{v_n}}_{\text{exploitation}} + \underbrace{c\sqrt{\frac{\ln(v_p)}{v_n}}}_{\text{exploration}} \tag{3}$$

The first term is simply the win rate: how often has this move led to wins? The second term grows larger for less-visited nodes, encouraging exploration. As a node accumulates visits, the exploration bonus shrinks and the win rate dominates. The constant $c = \sqrt{2} \approx 1.414$ controls the exploration/exploitation balance.

6.3.2 Expansion Phase

When a non-terminal node with untried moves is reached, one child is added. Rather than selecting moves randomly, we employ *threat-aware move ordering*: before any expansion occurs, all legal moves are scored and sorted by their tactical value. The scoring function evaluates each potential move by:

1. Checking if the move wins immediately (highest priority)
2. Checking if it blocks an opponent's winning move
3. Counting the threats created (three-in-a-rows, open patterns)
4. Evaluating center proximity as a tiebreaker

This ordering ensures that tactically promising moves are explored first, significantly reducing the number of simulations needed to identify strong candidates. The move ordering persists in each node's untried move list, so expansion naturally prioritizes high-value branches.

6.3.3 Simulation Phase

From the expanded node, a rollout plays to terminal state. Our "smart rollout" follows a decision hierarchy:

1. **Win:** Take immediate winning moves (100%)
2. **Block:** Block opponent winning threats (100%)
3. **Threat:** With 70% probability, maximize threat creation
4. **Random:** With 65% probability prefer center; otherwise random

This maintains statistical validity while capturing obvious tactical moves.

6.3.4 Backpropagation Phase

Results propagate up the tree: win = 1.0, draw = 0.5, loss = 0.0. Each node updates its visit count and cumulative value.

6.4 RAVE Enhancement

6.4.1 The Cold Start Problem

Standard MCTS has a bootstrapping problem: every new node in the tree starts with zero visits and zero knowledge. When the algorithm first encounters a position, it has no information about which moves are good or bad. It must run many simulations before the statistics become meaningful enough to guide search effectively.

Consider a concrete example: suppose through hundreds of simulations, MCTS has learned that playing in column 3 tends to lead to wins. This knowledge exists in heavily-visited nodes near the root. But when the search reaches a new, unexplored position deeper in the tree, that node knows nothing about column 3. The algorithm must re-learn from scratch that column 3 is strong, even though this information already exists elsewhere in the tree.

This is wasteful. If a move is generally strong, we should be able to leverage that knowledge immediately rather than rediscovering it at every node.

6.4.2 The RAVE Insight

Rapid Action Value Estimation (RAVE) addresses this by sharing move quality information across the entire tree. The key insight is that a move's strength is often somewhat independent of the exact position. Column 3 being a strong move in one position suggests it might be strong in similar positions too.

RAVE works by tracking, for each node, how well each move performed in *any* simulation that passed through that node, regardless of when during the simulation that move was actually played. This creates a "global reputation" for each move:

- **Without RAVE:** Node B's child "column 3" has 0 visits, so UCT treats it as completely unknown. We must explore it blindly.
- **With RAVE:** "Column 3 has won 70% of games across the whole tree. Even though we have never tried it from this exact position, let's start with that as a prior estimate instead of zero."

RAVE essentially provides informed initial guesses for unexplored moves, allowing the search to make smarter choices earlier.

6.4.3 RAVE Formulation

The RAVE value for action a at node n is simply the win rate of that action across all simulations through the node:

$$\text{RAVE}_n(a) = \frac{\tilde{w}_{n,a}}{\tilde{v}_{n,a}} \tag{4}$$

where $\tilde{w}_{n,a}$ is the cumulative value and $\tilde{v}_{n,a}$ is the count of simulations where action a appeared.

The final node score blends UCT (position-specific knowledge) with RAVE (tree-wide knowledge):

$$\text{Score}(n) = (1 - \beta) \cdot \text{UCT}(n) + \beta \cdot \text{RAVE}_n(a_n) \tag{5}$$

The blending parameter β starts high and decays as the node accumulates its own statistics:

$$\beta = \frac{\tilde{v}_{n,a}}{v_n + \tilde{v}_{n,a} + 4 \cdot v_n \cdot \tilde{v}_{n,a}/K} \tag{6}$$

with $K = 500$ as the decay constant. Early on (few visits), β is large and RAVE dominates, providing informed guesses. As the node gains experience, β shrinks and position-specific UCT statistics take over. This gradual handoff ensures we use global knowledge when local knowledge is scarce, but trust local experience once we have it.

6.5 Tactical Threat Detection

6.5.1 Why Random Rollouts Are Not Enough

MCTS relies on random game simulations to estimate move quality. This works remarkably well for strategic evaluation, but it has a critical weakness: random play can miss forced tactical sequences.

Imagine the opponent has three pieces in a row and will win on their next turn unless blocked. In a random rollout, the blocking move has only a 1-in-42 chance of being selected. The other 41 simulations from this position will show losses, but only because the rollout randomly failed to make the obvious defensive move. The resulting statistics suggest the position is bad, when really it is fine if we just block.

Worse, consider a position where we can win immediately. A random rollout might not find the winning move, play something else, and eventually lose. The statistics would undervalue a position that is actually a guaranteed win.

These "must-do" moves (winning moves, forced blocks) should not be left to chance. A human player would never miss them, and neither should the AI.

6.5.2 Pre-Search Tactical Shortcuts

Before MCTS begins its search, a deterministic tactical layer checks for critical positions and returns immediately if a forced move exists:

1. **Immediate Win (Priority 1):** If any move wins the game, return it immediately. No search needed.
2. **Forced Block (Priority 2):** If the opponent has a winning threat, block it. No search needed.
3. **Fork Creation (Priority 3):** If a move creates multiple simultaneous winning threats (a "fork"), play it. Forks force wins because the opponent cannot block both.
4. **Block Opponent Fork (Priority 4):** Prevent the opponent from creating a fork on their next turn.

These checks are deterministic and guarantee that the AI never misses an immediate winning move or fails to block a direct threat. Only when no forced tactical move exists does the position proceed to MCTS for strategic evaluation.

6.5.3 Smart Rollout Policy

A separate tactical mechanism operates *within* the MCTS simulations themselves. The "smart rollout" described in Section 6.3.3 applies similar heuristics during random playouts, but with probabilistic elements (70% threat check, 65% center bias) to maintain statistical diversity. This improves rollout quality while preserving the randomness MCTS requires for accurate value estimation.

6.6 Threat Detection Algorithm

Threats are identified by scanning all possible four-cell lines:

- **Three-in-a-row:** Three pieces plus one empty (immediate threat)
- **Open three:** Three-in-a-row with both ends empty
- **Two-in-a-row:** Two pieces plus two empty (developing threat)

Critically, threats only count if the empty cell has support below (respecting gravity). The scoring weights:

Listing 1: Threat Scoring Weights

```
three_in_a_row:      100 points
open_three_in_a_row: 500 points  # Nearly winning
two_in_a_row:         10 points
open_two_in_a_row:    30 points
double_threat:      5000 points  # Fork - forces win
```

6.7 Numba JIT Optimization

6.7.1 The Performance Problem

MCTS requires running thousands of simulations per move to achieve strong play. Each simulation involves dozens of move generations, board updates, and win checks. In pure Python, interpreter overhead makes this prohibitively slow: approximately 25 simulations per second, meaning a 1,000-simulation search would take 40 seconds. This is far too slow for interactive gameplay.

6.7.2 The Solution: JIT Compilation

Numba's Just-In-Time compiler translates Python and NumPy code into optimized machine code at runtime. The first call to a JIT-compiled function incurs compilation overhead, but subsequent calls run at near-C speeds. This provides a 24x speedup, bringing performance to 619 simulations per second.

To maximize Numba's effectiveness, the code follows several patterns:

- **1D Array Representation:** Flat arrays avoid multi-dimensional indexing overhead and enable contiguous memory access

- **Pre-allocated Arrays:** All temporary arrays are allocated once outside hot loops, eliminating repeated memory allocation
- **Custom RNG:** NumPy's random functions are not Numba-compatible, so we use a simple Linear Congruential Generator
- **Explicit Types:** Consistent use of specific dtypes (int32, float64, uint8) enables optimal compilation

The custom LCG uses the classic formula:

$$\text{state}_{n+1} = (\text{state}_n \times 1103515245 + 12345) \mod 2^{31} \tag{7}$$

While not cryptographically secure, this RNG is fast and provides sufficient randomness for game simulations.

6.8 Tree Reuse Between Moves

A naive MCTS implementation discards the entire search tree after each move and rebuilds from scratch. This wastes the statistical knowledge accumulated during previous searches. Our implementation preserves subtrees across moves through a technique called *tree reuse*.

When the AI makes a move, the corresponding child node becomes the new root. All statistics from that subtree, including visit counts, win rates, and RAVE values, are retained. When the opponent makes their move, the tree is again advanced to the corresponding child.

This provides two benefits:

1. **Knowledge preservation:** If the AI considered opponent responses during its previous search, those statistics carry forward rather than being recomputed.
2. **Faster convergence:** Subsequent searches start with informed priors rather than zero knowledge, allowing earlier focus on promising variations.

When a move falls outside the explored tree (the opponent played something unexpected), the system gracefully falls back to creating a fresh tree from the current position.

6.9 Search Optimizations

Two additional optimizations improve search efficiency:

6.9.1 Early Exit

During search, if one move accumulates an overwhelming share of visits (85% or more of total simulations), the search terminates early. At this point, additional simulations are unlikely to change the outcome, and the computational budget is better saved for future moves. This optimization is particularly effective in positions with a single clearly-best move.

6.9.2 Adaptive Time Budgeting

Not all positions deserve equal thinking time. The AI adjusts its time budget based on position characteristics:

- **Critical positions (1.5x time):** When the opponent has open three-in-a-rows, the position requires careful defense. Missing a block here loses immediately, so extra time is allocated.
- **Opening positions (0.4x time):** In the early game (less than 15% of cells occupied), strategic differences between moves are smaller and the position is far from tactical crisis. Less time is needed.
- **Simplified positions (0.5x time):** Late in the game (more than 60% filled) or when few moves remain (10 or fewer), the reduced branching factor means positions are easier to evaluate accurately with fewer simulations.
- **Complex midgame (up to 1.3x time):** Positions with many valid moves and active piece play receive proportionally more time based on the branching factor.

This adaptive approach concentrates computational effort where it matters most: critical defensive moments and complex tactical positions.

7 Evaluation

7.1 Experimental Setup

Experiments ran on Python 3.11 with Numba 0.58. The AI was tested at four difficulty levels:

Table 3: Difficulty Settings

Difficulty	Simulations per Move
Easy	200
Medium	500
Hard	1,000
Nightmare	2,000

7.2 Hypothesis 1: MCTS vs. Random Opponent

Hypothesis: MCTS will defeat a purely random opponent.

Method: 20 games with MCTS (1,000 simulations) against random move selection, alternating which player moves first.

Result: 100% win rate (20 wins, 0 losses, 0 draws).

Analysis: The random opponent made no effort to block threats, allowing MCTS to establish dominant positions within 10-15 moves.

7.3 Hypothesis 2: MCTS vs. Greedy Lookahead

Hypothesis: MCTS will outperform a greedy one-step heuristic.

Method: 20 games against a greedy AI that takes winning moves, blocks opponent wins, and otherwise plays center-biased moves, alternating which player moves first.

Result: 100% win rate for MCTS (20 wins, 0 losses, 0 draws).

Analysis: The greedy opponent fell into traps where MCTS sacrificed short-term evaluation for forcing sequences 3-5 moves ahead.

7.4 Hypothesis 3: Enhanced vs. Basic MCTS

Hypothesis: MCTS with tactical heuristics and RAVE outperforms basic random-rollout MCTS.

Method: 20 games between Enhanced and Basic MCTS, both at 500 simulations per move, alternating which player moves first.

Result: 100% win rate for Enhanced MCTS (20 wins, 0 losses, 0 draws).

7.5 Performance Benchmarks

Numba JIT provided substantial speedups:

Table 4: Performance: Pure Python vs. Numba JIT

Operation	Pure Python	Numba JIT	Speedup
Win Check	$\sim$100 μs	$\sim$0.8 μs	**125**$\times$
Full Rollout	$\sim$9.6 ms	$\sim$0.4 ms	**24**$\times$
Simulations/sec	$\sim$25	$\sim$619	**24**$\times$

At 619 simulations/second, 1,000 simulations (Hard difficulty) complete in approximately 1.6 seconds.

8 Conclusion and Future Work

8.1 Summary

We presented a Monte Carlo Tree Search implementation for 3D Connect Four that addresses the 10^{120} state space through intelligent sampling rather than exhaustive search. By combining UCT selection, RAVE estimation, tactical threat detection, and Numba JIT optimization, the system achieves strong play at interactive speeds.

Key findings:

- MCTS provides effective decision-making without exhaustive search
- Tactical heuristics catch critical positions that random sampling might miss
- RAVE accelerates convergence in early tree nodes
- JIT compilation is essential for real-time response

8.2 Future Work

1. **Neural Network Integration:** Train a network through self-play to replace rollouts with instant position evaluation
2. **Transposition Tables:** Recognize equivalent states reached through different move orders

3. **WebGL Deployment:** Enable browser-based play without installation
4. **Opening Book:** Pre-compute strong opening moves
5. **Parallel MCTS:** Leverage multi-core processors

References

[1] Silver, D., Hubert, T., Schrittwieser, J., et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv preprint arXiv:1712.01815.

[2] Kocsis, L., & Szepesvári, C. (2006). *Bandit based Monte-Carlo Planning*. In European Conference on Machine Learning (pp. 282-293). Springer.

[3] Tromp, J. (1988-present). *Fhourstones: A Connect-4 Benchmark*. `https://tromp.github.io/c4/c4.html`

[4] Gelly, S., & Silver, D. (2007). *Combining Online and Offline Knowledge in UCT*. In Proceedings of the 24th International Conference on Machine Learning (pp. 273-280).

[5] Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. In International Conference on Computers and Games (pp. 72-83). Springer.